



Université Abdelmalek Essaâdi

L'Ecole Nationale des Sciences Appliquées d'Al-Hoceima

Filière : Transformation Digitale et Intelligence Artificielle



Compte Rendu de TP2

Applications Web Avancées avec Java et Spring



Encadré par : Pr. M. CHERRADI

Réalisé par : Nada EL-KHAMLI

Année universitaire : 2025 / 2026

Introduction :

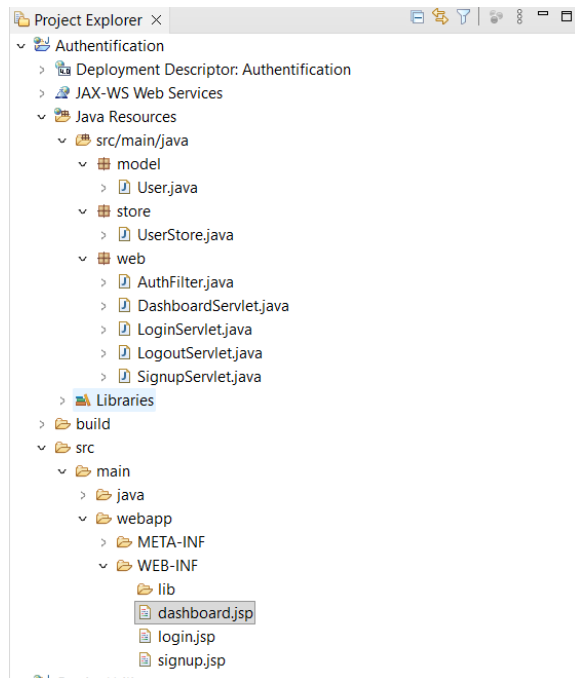
Dans le cadre du module Java EE, nous avons réalisé une mini application portant sur la conception et le développement d'un système d'authentification web.

L'objectif principal de cette application est de permettre à un utilisateur de créer un compte, se connecter, accéder à un espace sécurisé (dashboard) et se déconnecter. Ce projet a été réalisé en utilisant les technologies Java EE, notamment les Servlets, JSP et Filters, tout en respectant l'architecture MVC.

1. Architecture du projet :

L'application suit le modèle MVC :

- **Modèle (Model) :**
 - User.java
 - UserStore.java
- **Contrôleur (Controller) :**
 - SignupServlet.java
 - LoginServlet.java
 - LogoutServlet.java
 - DashboardServlet.java
- **Filtre (Sécurité) :**
 - AuthFilter.java
- **Vue (View) :**
 - login.jsp
 - signup.jsp
 - dashboard.jsp



2. Description des composants :

User et UserStore :

- La classe User représente un utilisateur avec ses informations (username, password)

```
User.java x
package Authentication/src/main/java/model/User.java

public class User {
    private String username;
    private String password;

    public User() {
    }

    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

- La classe UserStore permet de stocker les utilisateurs en mémoire à l'aide d'une structure Map.

```
UserStore.java x
package store;

import java.util.HashMap;

public class UserStore {

    private static final Map<String, User> users = new HashMap<>();

    static {
        users.put("admin", new User("admin", "1234"));
    }

    public static boolean addUser(User user) {
        if (users.containsKey(user.getUsername())) {
            return false;
        }
        users.put(user.getUsername(), user);
        return true;
    }

    public static User getUser(String username) {
        return users.get(username);
    }

    public static boolean exists(String username) {
        return users.containsKey(username);
    }

    public static boolean validateUser(String username, String password) {
        User user = users.get(username);
        return user != null && user.getPassword().equals(password);
    }
}
```

SignupServlet :

La servlet **SignupServlet**, accessible via l'URL `/signup`, est responsable de la gestion du processus d'inscription des utilisateurs.

La méthode `doGet()` permet d'afficher la page d'inscription (`signup.jsp`), tandis que la méthode `doPost()` traite les données soumises par le formulaire.

Elle assure la création d'un nouvel utilisateur après vérification de l'unicité du nom d'utilisateur. En cas de succès, l'utilisateur est redirigé vers la page de connexion, sinon un message d'erreur est affiché.

```
SignupServlet.java
package web;

import java.io.IOException;

@WebServlet("/signup")
public class SignupServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.getRequestDispatcher("/WEB-INF/signup.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        User user = new User(username, password);
        boolean added = UserStore.addUser(user);

        if (added) {
            request.setAttribute("successMessage", "Inscription réussie. Connectez-vous maintenant.");
            request.getRequestDispatcher("/login").forward(request, response);
        } else {
            request.setAttribute("errorMessage", "Ce nom d'utilisateur existe déjà.");
            request.getRequestDispatcher("/WEB-INF/signup").forward(request, response);
        }
    }
}
```

LoginServlet :

La servlet **LoginServlet**, accessible via l'URL `/login`, gère le processus d'authentification des utilisateurs.

La méthode `doGet()` affiche le formulaire de connexion (`login.jsp`), tandis que la méthode `doPost()` vérifie les identifiants saisis.

En cas d'authentification réussie, une session est créée pour l'utilisateur, qui est ensuite redirigé vers la page sécurisée du dashboard. Dans le cas contraire, un message d'erreur est affiché.

```
LoginServlet.java
package web;

import java.io.IOException;

@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.getRequestDispatcher("/WEB-INF/login.jsp").forward(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if (UserStore.validateUser(username, password)) {
            HttpSession session = request.getSession();
            session.setAttribute("user", username);
            response.sendRedirect(request.getContextPath() + "/dashboard");
        } else {
            request.setAttribute("errorMessage", "Nom d'utilisateur ou mot de passe incorrect.");
            request.getRequestDispatcher("/WEB-INF/login.jsp").forward(request, response);
        }
    }
}
```

DashboardServlet :

La servlet **DashboardServlet**, accessible via l'URL `/dashboard`, permet d'afficher la page principale de l'application après authentification.

Cette page est protégée et ne peut être accessible qu'aux utilisateurs authentifiés. Elle présente un message de bienvenue personnalisé ainsi qu'un accès à la déconnexion.

```
DashboardServlet.java x
package web;

import java.io.IOException;

@WebServlet("/dashboard")
public class DashboardServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        request.getRequestDispatcher("/WEB-INF/dashboard.jsp").forward(request, response);
    }
}
```

LogoutServlet :

La servlet **LogoutServlet**, accessible via l'URL `/logout`, est chargée de gérer la déconnexion de l'utilisateur.

Elle invalide la session en cours afin de supprimer les informations d'authentification, puis redirige l'utilisateur vers la page de connexion.

```
LogoutServlet.java x
package web;

import java.io.IOException;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        HttpSession session = request.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        request.getRequestDispatcher("/login").forward(request, response);
    }
}
```

AuthFilter :

Le filtre **AuthFilter** est utilisé pour sécuriser l'accès à la page du dashboard.

Il intercepte les requêtes destinées à l'URL `/dashboard` et vérifie si l'utilisateur est authentifié à travers la session. Si aucune session valide n'est trouvée, l'utilisateur est automatiquement redirigé vers la page de connexion. Dans le cas contraire, l'accès à la ressource demandée est autorisé.

```

*AuthFilter.java X
package web;

import java.io.IOException;

@WebFilter("/dashboard")
public class AuthFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);
        boolean isLoggedIn = (session != null && session.getAttribute("user") != null);

        if (isLoggedIn) {
            chain.doFilter(request, response);
        } else {
            res.sendRedirect(req.getContextPath() + "/login");
        }
    }
}

```

Les pages JSP :

L'application comporte plusieurs pages JSP assurant l'interface utilisateur.

- La page **login.jsp** permet à l'utilisateur de saisir ses identifiants afin d'accéder à l'application. Elle offre également un retour visuel à travers l'affichage de messages d'erreur ou de succès selon le résultat de l'authentification.

```

login.jsp
<% page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<html>
<head>
<meta charset="UTF-8">
<title>login</title>
<style>
body {
font-family: Arial, sans-serif;
background: #f2f2f2;
}
div {
width: 300px;
margin: 20px auto;
background: #fff;
padding: 10px;
border-radius: 10px;
box-shadow: 0 0 10px #ccc;
}
input, button {
width: 100%;
padding: 5px;
margin-top: 10px;
}
.error {
color: red;
}
.success {
color: green;
}
a {
text-decoration: none;
}
</style>
</head>
<body>
<div class="box">
<div>Connexion</div>
<div>
<% String error = (String) request.getAttribute("errorMessage"); %>
<% String success = (String) request.getAttribute("successMessage"); %>
<% if (error != null) { %>
<p class="error"><%= error %></p>
<% } %>
<% if (success != null) { %>
<p class="success"><%= success %></p>
<% } %>
<form action="${pageContext.request.contextPath}/login" method="post">
<input type="text" name="username" placeholder="Nom d'utilisateur" required>
<input type="password" name="password" placeholder="Mot de passe" required>
<button type="submit">Se connecter</button>
</form>
<p>Vous n'avez pas de compte ? <a href="${pageContext.request.contextPath}/signup">S'inscrire</a></p>
</div>
</body>
</html>

```

- La page **signup.jsp** est dédiée à l'inscription des nouveaux utilisateurs. Elle contient un formulaire permettant la saisie des informations nécessaires à la création d'un compte, tout en assurant une validation des données.

```

signup.jsp x
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Signup</title>
<style>
body {
font-family: Arial, sans-serif;
background: #eef2f7;
}
.box {
width: 350px;
margin: 100px auto;
background: #fff;
padding: 25px;
border-radius: 10px;
box-shadow: 0 0 10px #ccc;
}
input, button {
width: 100%;
padding: 10px;
margin-top: 10px;
}
.error {
color: #f00;
}
a {
text-decoration: none;
}
</style>
</head>
<body>
<div class="box">
<h2>Inscription</h2>
<% String error = (String) request.getAttribute("errorMessage"); %>
<% if (error != null) { %>
<p class="error"><%= error %></p>
<% } %>
<form action="${pageContext.request.contextPath}/signup" method="post">
<input type="text" name="username" placeholder="Nom d'utilisateur" required>
<input type="password" name="password" placeholder="Mot de passe" required>
<button type="submit">S'inscrire</button>
</form>
<p>Vous avez déjà un compte ? <a href="${pageContext.request.contextPath}/login">Se connecter</a></p>
</div>
</body>
</html>

```

- La page **dashboard.jsp** constitue l'interface principale après connexion. Elle affiche un message de bienvenue personnalisé ainsi qu'un lien permettant à l'utilisateur de se déconnecter.

```

signup.jsp dashboard.jsp x
<%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<% String user = (String) session.getAttribute("user"); %>
<html>
<head>
<meta charset="UTF-8">
<title>Dashboard</title>
<style>
body {
font-family: Arial, sans-serif;
background: #f4f6f8;
text-align: center;
margin-top: 100px;
}
.box {
display: inline-block;
background: #fff;
padding: 30px;
border-radius: 10px;
box-shadow: 0 0 10px #ccc;
}
a {
display: inline-block;
margin-top: 20px;
text-decoration: none;
background: #f00;
color: #fff;
padding: 10px 15px;
border-radius: 5px;
}
</style>
</head>
<body>
<div class="box">
<h1>Bienvenue <%= user %></h1>
<p>Vous êtes connecté avec succès.</p>
<a href="${pageContext.request.contextPath}/logout">Se déconnecter</a>
</div>
</body>
</html>

```

3. Fonctionnement global :

Le fonctionnement de l'application repose sur une succession d'étapes assurant une gestion complète du cycle d'authentification.

Dans un premier temps, l'utilisateur accède à la page d'inscription afin de créer un compte. Une fois l'inscription effectuée, il est redirigé vers la page de connexion où il saisit ses identifiants. Après vérification, une session est créée pour l'utilisateur authentifié.

L'accès au dashboard est ensuite contrôlé par un filtre qui vérifie l'existence de la session. Si l'utilisateur est authentifié, il peut accéder à la page sécurisée. Dans le cas contraire, il est redirigé vers la page de connexion. Enfin, l'utilisateur peut mettre fin à sa session en se déconnectant.

4. Sécurité :

Plusieurs mécanismes ont été mis en place afin d'assurer la sécurité de l'application.

Tout d'abord, les pages JSP sont placées dans le répertoire `/WEB-INF`, ce qui empêche leur accès direct depuis le navigateur. Cette organisation garantit que toutes les requêtes passent obligatoirement par les servlets.

Ensuite, un filtre d'authentification a été implémenté afin de contrôler l'accès aux ressources sensibles. Ce filtre permet de vérifier l'état de la session avant d'autoriser ou non l'accès à certaines pages.

Enfin, la gestion des sessions est assurée à l'aide de l'objet `HttpSession`, permettant de maintenir l'état de connexion de l'utilisateur.

4. Solutions apportées :

Afin de résoudre les problèmes rencontrés, plusieurs solutions ont été mises en œuvre.

Une distinction claire a été établie entre l'utilisation de `forward`, destiné à l'affichage des pages JSP, et `sendRedirect`, utilisé pour les redirections vers les servlets.

L'utilisation de l'expression `${pageContext.request.contextPath}` a permis de corriger les erreurs liées aux chemins d'accès.

Par ailleurs, la mise en place d'un filtre d'authentification a renforcé la sécurité de l'application. Enfin, des opérations de nettoyage et de redéploiement du serveur Tomcat ont permis de résoudre les problèmes liés au cache et au déploiement.

5. Conclusion :

Ce mini projet a permis de mettre en pratique les concepts fondamentaux du développement web avec Java EE.

Il a notamment permis de comprendre le rôle des servlets dans le traitement des requêtes, l'utilisation des JSP pour l'affichage, ainsi que la gestion des sessions et la sécurisation des ressources à l'aide des filtres.

Ce travail constitue une base solide pour le développement d'applications web plus complexes intégrant des fonctionnalités avancées.

6. Perspectives d'amélioration :

Plusieurs améliorations peuvent être envisagées afin d'enrichir cette application.

Il serait notamment pertinent d'intégrer une base de données telle que MySQL afin de persister les données des utilisateurs. Le chiffrement des mots de passe permettrait également d'améliorer la sécurité.

L'ajout de rôles utilisateurs, ainsi que l'amélioration de l'interface graphique à l'aide de frameworks modernes, représentent également des pistes intéressantes. Enfin, la mise en place d'une validation côté client permettrait d'améliorer l'expérience utilisateur.